

Dos formas de definir a los agentes

En los módulos 1 a 3, creaste agentes usando **código de Python** en `agent.py`:

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor_agent',
    description='Ayuda a los estudiantes a aprender álgebra',
    instruction='Eres un tutor de matemáticas paciente...'
)
```

Este método funciona muy bien para el desarrollo. Pero el ADK también admite la definición de agentes con **archivos de configuración YAML**, que puede ser más sencillo en estos casos:

- Para que puedan editarlos quienes no sean programadores
- Control de versiones y uso compartido
- Experimentación rápida
- Separación de la configuración del código

En este módulo, se presenta **Agent Config**, que es el enfoque basado en YAML para crear agentes del ADK.

Nota sobre los lenguajes admitidos

Importante: El ADK admite varios lenguajes de programación.

- **Python:** Compatibilidad total de Agent Config de YAML y el código de Python.
- **Java:** Admite agentes basados en código de Python (consulta la [Guía de inicio rápido de Java](#)).

Actualmente, Agent Config (YAML) solo está disponible en Python. Los documentos del ADK brindan la siguiente información:

“Lenguaje de programación: La función de Agent Config actualmente solo admite código de Python para herramientas y otras funciones que requieran código de programación”.

Este módulo se enfoca en la función Agent Config de YAML, que está disponible cuando se usa el ADK con Python.

2 métodos para definir agentes

Referencia: [Documentos del ADK: Agent Config](#)

Método 1: Código de Python (módulos 1 a 3)

Creado con: `adk create my_agent`

Genera:

```
my_agent/
├── agent.py      # El código de Python que define tu agente
├── __init__.py
└── .env
```

agent.py:

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='assistant_agent',
    description='Un asistente útil',
    instruction='Eres un asistente útil.'
)
```

Método 2: Configuración de YAML (este módulo)

Creado con: `adk create --type=config my_agent`

Genera:

```
my_agent/
├── root_agent.yaml # Archivo de configuración YAML
└── .env
```

root_agent.yaml:

```
name: assistant_agent
model: gemini-2.5-flash
description: Un asistente útil
instruction: Eres un asistente útil.
```

La documentación del ADK brinda la siguiente información:

“La función Agent Config del ADK te permite crear un flujo de trabajo de ADK sin escribir código. Una configuración de agente usa un archivo de texto en formato YAML con una breve descripción del agente”.

Crea un agente basado en YAML

Referencia: [Documentos del ADK - Agent Config: Crea un agente](#)

Ahora, creemos tu primer agente basado en YAML paso a paso. El proceso es similar a lo que aprendiste en el módulo 1, pero con una diferencia clave: en lugar de generar código de Python, generaremos un archivo de configuración YAML.

Paso 1: Crea el proyecto del agente

Ejecuta el siguiente comando para crear un agente basado en configuración:

```
adk create --type=config my_config_agent
```

Observa el indicador `--type=config`. Esto le indica al ADK que debe crear un agente basado en YAML en lugar del agente predeterminado basado en código de Python.

La documentación del ADK brinda la siguiente información:

"Este comando genera una carpeta `my_agent/`, que contiene un archivo `root_agent.yaml` y un archivo `.env`".

Verás una nueva carpeta `my_config_agent/` con estos elementos:

- `root_agent.yaml`: La configuración de tu agente (en lugar de `agent.py`)
- `.env`: Variables de entorno para claves de API (igual que en el módulo 1)

Paso 2: Configura tu clave de API

Al igual que en el módulo 1, debes configurar tu archivo `.env` con tu clave de API. El proceso es idéntico: Abre `my_config_agent/.env` y agrega tu `GOOGLE_API_KEY` como lo hiciste en el módulo 1, paso 7.

Si necesita un repaso, consulta el [Módulo 1, paso 7: Configura tu clave de API](#).

Paso 3: Edita la configuración de YAML

Aquí es donde las cosas se ponen interesantes. En lugar de escribir código de Python, editarás un archivo YAML.

Abre `root_agent.yaml` en tu editor de texto. El archivo predeterminado generado debería verse de la siguiente manera:

```
# yaml-language-server:
$schema=https://raw.githubusercontent.com/google/adk-python/refs/heads/main/src/google/adk/agents/config_schemas/AgentConfig.json
name: assistant_agent
model: gemini-2.5-flash
description: Un agente asistente que puede responder las preguntas de los usuarios.
Instrucción: Eres un agente que ayuda a responder diversas preguntas de los usuarios.
```

Esta es una configuración básica del agente con cuatro parámetros clave. Observa lo limpio y legible que es el archivo YAML en comparación con el código de Python. No hay importaciones ni definiciones de funciones, solo pares clave-valor sencillos.

Personalicemos el código para crear el mismo agente tutor de matemáticas que creaste en el módulo 2. Edita tu archivo `root_agent.yaml` para que se vea así:

```
# yaml-language-server:
$schema=https://raw.githubusercontent.com/google/adk-python/refs/heads/main/src/google
/adk/agents/config_schemas/AgentConfig.json
name: math_tutor_agent
model: gemini-2.5-flash
description: Ayuda a los estudiantes a aprender álgebra guiándolos a través de pasos
de resolución de problemas.
instruction: |
  Eres un tutor de álgebra paciente y motivador.

  Este es tu enfoque de enseñanza:
  1. Cuando un estudiante pregunta, primero debes entender cuál es la dificultad
  2. Divide el problema en pasos más pequeños y manejables
  3. Guíalo para que pueda descubrir la respuesta, en lugar de dársela directamente
  4. Ofrece refuerzos positivos por su esfuerzo y progreso
  5. Usa un lenguaje sencillo y evita la jerga

  Siempre mantén un tono comprensivo y paciente. Aprender lleva tiempo y cada
  pregunta es una oportunidad para crecer.
```

Comprende la sintaxis YAML:

La barra vertical `|` después del parámetro `instruction:` le indica a YAML que todo lo que sigue es texto de varias líneas. Esto es ideal para escribir instrucciones detalladas de varias líneas. Por lo general, no se necesitan comillas ni secuencias de escape especiales. Simplemente escribe tus instrucciones de forma natural.

Cada parámetro se relaciona directamente con lo que aprendiste en el módulo 2:

- `name:` El identificador único del agente.
- `model:` Qué LLM utilizar (`gemini-2.5-flash`).
- `description:` Qué hace el agente (para que otros agentes lo entiendan).
- `instruction:` Cómo debe comportarse el agente (la parte más importante).

Paso 4: Ejecuta tu agente basado en YAML

Esta es la mejor parte: **Ejecutar un agente basado en YAML es exactamente lo mismo que ejecutar un agente basado en Python.** Todos los comandos que aprendiste en el módulo 3 funcionan de manera idéntica.

Desde el directorio de tu espacio de trabajo (la carpeta superior que contiene `my_config_agent`), ejecuta el siguiente código:

```
adk web my_config_agent
```

Funciona de la misma manera que en los módulos 1 y 2. El ADK carga tu archivo `root_agent.yaml`, crea el agente según esa configuración e inicia la interfaz web.

La documentación del ADK brinda la siguiente información:

"Puedes ejecutar el agente definido con Agent Config-... **adk web**. Ejecuta la interfaz de usuario web para tu agente".

Una vez que se abra la interfaz web, interactúa con tu agente tutor de matemáticas tal como lo hiciste en el módulo 2. Pregunta: "¿Cómo resuelvo $2x + 5 = 13$?" y observa cómo te guía a través del problema paso a paso.

YAML en comparación con Python: Cuándo usar cada uno

Ahora que viste ambos enfoques, quizás te preguntes: "¿Cuándo debería usar la configuración YAML en lugar del código de Python?". Ambos métodos crean agentes que funcionan de manera idéntica, pero son adecuados para situaciones diferentes.

Aspecto	Configuración de YAML	Código de Python
Facilidad de edición	✓ Muy fácil (archivo de texto)	⚠ Requiere conocimientos de programación
Casos de uso indicados	Agentes simples, prototipos rápidos	Agentes complejos, lógica personalizada
Control de versiones	✓ Diferencias claras	⚠ Cambios en el código
Flexibilidad	⚠ Limitada a las opciones de configuración	✓ Control programático completo
Usuarios que no son programadores	✓ Pueden editar	✗ Necesitan habilidades de programación
Funciones avanzadas	⚠ Disponibles próximamente	✓ Acceso total

Elige YAML en los siguientes casos:

Quieres simplicidad y accesibilidad. La configuración de YAML es perfecta para crear agentes sencillos y conservar la simpleza. Las personas que no son programadoras pueden editar fácilmente un archivo YAML para ajustar el comportamiento del agente sin modificar nada del código, lo que es ideal para la colaboración entre equipos en los que no todos tienen experiencia en programación.

Necesitas experimentación rápida. ¿Quieres probar diferentes instrucciones o cambiar entre modelos? Solo edita el archivo YAML y reinicia tu agente. No es necesario navegar por importaciones y definiciones de clases de Python.

La configuración debe estar separada del código. YAML mantiene la configuración de tu agente totalmente separada de cualquier lógica personalizada que puedas agregar más

adelante. Esto facilita la administración de diferentes parámetros de configuración para entornos de desarrollo, etapas de prueba y producción.

Elige Python en los siguientes casos:

Crearás sistemas complejos de múltiples agentes. Como aprenderás en el curso 7, las arquitecturas sofisticadas de múltiples agentes con delegación, agentes de flujo de trabajo y organización personalizada requieren todo el poder del código de Python. YAML está limitado a opciones de configuración básicas.

Necesitas herramientas personalizadas y devoluciones de llamadas. En los cursos 4 y 5, aprenderás a implementar devoluciones de llamadas y agregar herramientas personalizadas. Estas funciones avanzadas requieren código de Python y no se pueden expresar completamente solo en YAML.

Quieres control programático. A veces es necesario generar parámetros de configuración de agentes dinámicamente, modificar el comportamiento en función de las condiciones del entorno de ejecución o implementar una integración estrecha con otras bibliotecas de Python. El código de Python te brinda esta flexibilidad.

Ejemplo: Comparación en paralelo

Para comprender realmente la equivalencia entre YAML y Python, analicemos al mismo agente simple definido con cada enfoque. Este agente de saludo tiene los mismos cuatro parámetros que hemos estado usando en este curso.

Código de Python (agent.py):

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='greeting_agent',
    description='Un agente de bienvenida amable',
    instruction='Saluda a los usuarios de forma cálida y profesional.'
)
```

Configuración de YAML (root_agent.yaml):

```
name: greeting_agent
model: gemini-2.5-flash
description: Un agente de bienvenida amable
instruction: Saluda a los usuarios de forma cálida y profesional.
```

El resultado: Ambos producen exactamente el mismo comportamiento del agente. Cuando

ejecutas cualquiera de las versiones con `adk web`, obtendrás un agente de bienvenida idéntico. La versión YAML es más limpia y fácil de leer; no hay importaciones, paréntesis ni comillas (en la mayoría de los casos), solo una configuración directa.

Objetivos logrados

Felicitaciones, ahora aprendiste que hay **dos formas de definir agentes en el ADK** y ambos son enfoques igualmente válidos que dependen de tus necesidades.

Comenzaste con el código de Python en los módulos 1 a 3, en los que creaste agentes escribiendo archivos `agent.py` con importaciones y la creación de instancias de clase. Te proporcionó experiencia práctica con la estructura fundamental de los agentes.

Luego, descubriste la configuración YAML, una forma más sencilla y sin código de definir agentes. Aprendiste que `adk create --type=config` genera un archivo `root_agent.yaml` en lugar de código de Python, lo que hace que la creación de agentes sea accesible para quienes no son programadores.

Lo más importante es que debes comprender que ambos enfoques producen agentes idénticos. Ya sea que definas tu tutor de matemáticas en Python o YAML, se comportará exactamente de la misma manera. Los comandos `adk web`, `adk run` y `adk api_server` funcionan en ambos casos.

También aprendiste en qué casos elegir cada método: YAML para la simplicidad y la creación rápida de prototipos, y Python para sistemas complejos de múltiples agentes y funciones avanzadas que aprenderás en cursos futuros.

Por último, sabes que **Agent Config actualmente solo es compatible con Python**, aunque el ADK en sí admite varios lenguajes de programación, incluido Java. A medida que el framework evoluciona, la compatibilidad con YAML puede expandirse a otros lenguajes.

Conclusiones principales

El ADK te brinda flexibilidad para definir agentes. No tienes que escribir código si no quieres. La configuración de YAML proporciona una alternativa limpia y accesible para agentes simples.

La **configuración de YAML es ideal** cuando necesitas experimentar rápidamente, cuando los miembros no técnicos del equipo necesitan ajustar el comportamiento del agente o cuando quieres una separación clara entre la configuración y el código. Puedes editar un archivo YAML, guardarlo y reiniciar tu agente sin necesidad de programación.

El código de Python se vuelve esencial cuando tus agentes se vuelven más sofisticados. Cuando comiences a agregar herramientas personalizadas (curso 4), implementar devoluciones de llamadas (curso 5) o crear sistemas de múltiples agentes (curso 7), necesitarás toda la potencia de Python. Pero siempre puedes comenzar con YAML y migrar a Python cuando sea necesario.

Ambos métodos son elementos destacados en el ADK. No hay una opción “correcta” o “incorrecta”. Elige el enfoque que mejor se adapte a tus necesidades actuales. Recuerda que el equipo de ADK está desarrollando activamente la función Agent Config y agradece los comentarios.

La documentación del ADK brinda la siguiente información:

“La función Agent Config es experimental y tiene algunas limitaciones conocidas. Nos encantaría recibir tus comentarios”.